

MARLET: Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring

Anonymous Authors

Abstract—Large language models can generate fluent tutoring responses, but they need grounding that respects both instructional evidence and the learner’s current state. Classical retrieval-augmented generation (RAG) grounds generation through a fixed retrieve-then-read controller: the query is matched against a corpus, and the generator reads the returned passages. That design is often insufficient for tutoring, where the same surface question can require different evidence, scaffolding, or rubric constraints for different learners. We propose MARLET (Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring), a framework that places a supervisory router, planner, diagnoser, rubric module, retriever, tutor, and critic around a fixed downstream tutor. The framework first builds a tutoring brief from the sample, inferred learning state, and answer criteria; retrieval is then invoked only when the route indicates that external evidence is needed. We validate the framework against classical RAG under matched backbones, corpus, reranker, and response budget on TutorEval and EduBench. MARLET improves fixed-tutor quality metrics over classical RAG while using fewer tokens; on EduBench it also reduces latency and backbone-call count. These results support the framework claim: for educational tutoring, grounding should be controlled by pedagogical state and task regime, not by the surface query alone.

I. Introduction

Large language models (LLMs) are increasingly used as educational tutors because they can generate explanations, diagnose errors, and provide feedback [1], [2]. However, an LLM tutor cannot rely only on parametric knowledge. Knowledge-grounded dialogue has long shown the value of external evidence [3]; educational responses similarly need grounding in textbook chapters, course notes, rubrics, worked examples, or learner profiles to avoid fluent but misaligned guidance.

Classical retrieval-augmented generation (RAG) has therefore become the default grounding mechanism [4]. In classical RAG, the query is embedded, the top- k most similar passages are retrieved, and the LLM generates a response conditioned on those passages. Thus, relevance depends heavily on retrieval quality and query representation [5].

Educational tutoring is more complex than factual question answering. A student’s question is often only a partial signal: the appropriate response may depend on level, goals, weak points, recent confusion, dialogue, and likely misconceptions. We call this the learner’s personalized learning state. For example, an ordinary-differential-equation query may require high-school pro-

cedural scaffolding or college-level modeling context. Therefore, tutoring retrieval must find evidence that is pedagogically appropriate for the learner, not merely topical.

Classical RAG can include explicit learner-state text when supplied, but appending learner state, dialogue, grading criteria, and task constraints forces one query embedding to compress topic, level, misconception, instructional goal, and response constraints. As constraints grow, the query can blur search intent and return evidence that satisfies some needs while missing others.

Therefore, we propose MARLET (Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring). Instead of letting the retriever act as the top-level controller, a supervisory router identifies the tutoring regime and decides whether retrieval is needed. A planner scopes search intent, a diagnoser summarizes visible learning state, and a rubric module states answer criteria before the fixed tutor writes.

The experiments validate the framework rather than present a model leaderboard. We compare with classical retrieve-then-read RAG while holding the backbone, corpus, reranker, response cap, tutor, and critic fixed. The baseline tests the standard design in which retrieval is driven before an explicit learner-state, plan, and rubric brief is constructed. Our validation on TutorEval and EduBench shows that the same tutor scores higher when fed context assembled by MARLET; on EduBench it also reduces latency and backbone calls.

Contributions. (i) We propose MARLET, a learning-state-aware multi-agent retrieval framework with a supervisory router, planner, diagnoser, rubric module, conditional retriever, tutor, and critic. (ii) We formalize the router and retrieval gate as transparent, training-free controls over learner state, rubric constraints, and external evidence. (iii) We validate against classical RAG on two educational benchmarks under shared backbone, corpus, reranker, and response budget.

II. Related Work

Classical RAG fixes retrieval as the first controller: the query is matched to passages, and generation conditions on the returned evidence [4]. Selective and corrective RAG methods improve this controller. Self-RAG learns when to retrieve, generate, and critique; CRAG evaluates retrieved documents and attempts correction when retrieval is unreliable; Adaptive-RAG and TARG vary

retrieval behavior by query complexity or retrieval utility [6]–[9]. RAG diagnostics further emphasize separating retrieval, ranking, and generation errors [10]. These methods address whether passages retrieved for a query are necessary, sufficient, or reliable, but they do not infer learner stage before retrieval, separate rubric constraints from topic evidence, or decide whether external passages are the right source of grounding for a tutoring decision.

Agentic RAG and multi-agent LLM systems move beyond a single retrieve-then-read step. ReAct, AutoGen, CAMEL, and MetaGPT show that tool use and role decomposition can improve complex task execution [11]–[14]. MA-RAG uses multiple agents for query disambiguation, evidence extraction, and answer synthesis on ambiguous or multi-hop QA tasks [15]. These systems demonstrate coordination benefits, but their roles are designed for general task execution or ambiguous question answering. They do not define pedagogical regimes, infer visible learner state, or bind the final response to rubric-style answer criteria, so the educational reason for retrieving, skipping retrieval, or prioritizing rubric context remains implicit.

Recent educational work shows that tutoring quality requires more than answer accuracy. AgentTutor builds multi-turn personalized learning with learner assessment and memory [2]; adaptivity studies show that LLM tutors are sensitive to omitted student-context features [16]; and AI-tutor evaluation emphasizes pedagogy, mistake diagnosis, guidance quality, and learner support [1], [17]–[20]. These works define tutoring requirements, but they do not formulate retrieval as learner-state-aware control: when to retrieve, what evidence to retrieve, and how much context to allocate to evidence rather than learner-state and rubric information.

III. Methodology

The proposed framework, MARLET, is a tutoring pipeline organized around a supervisory controller. For each sample, the controller reads the question, dialogue, context text, rubric items, and metadata, then assigns a pedagogical regime and decides which modules should run. The planner states the instructional goal and search intent, the diagnoser summarizes visible learning state, the rubric module lists answer criteria, and retrieval is invoked only when external evidence is needed. Fig. 1 sketches this pipeline.

The module boundaries come from observed benchmark patterns. TutorEval samples such as “Which is the fastest mode of heat transfer?” require the answer radiation, the explanation that no medium is required, and alignment with chapter evidence; a thermal-conductivity-unit sample requires isolating k and reasoning from SI units. These cases motivate a planner that narrows evidence search, a retriever that supplies chapter grounding, and a rubric module that preserves expected teaching points. EduBench supplies different

TABLE I
Framework components and responsibilities.

Component	Responsibility
Supervisor	Assigns pedagogical regime and activates retrieval, rubric, and critic switches.
Planner	Converts the tutoring request into an operating plan and up to three scoped retrieval queries.
Diagnoser	Infers visible learning state from the prompt and dialogue, including level and likely misconceptions.
Rubric module	Summarizes explicit rubric items or default answer-quality criteria.
Retriever	Runs only when requested by the route or fallback gate, then reranks and packs evidence.
Tutor/Critic	Generates the final response and checks evidence markers, rubric coverage, and pedagogy.

signals: a personalized-learning sample describes a grade-five Chinese learner who likes folklore but struggles with pronunciation, while an emotional-support sample describes anxiety about speaking in class. These motivate a diagnoser that captures visible learning state and affect before generation. EduBench grading and error-correction samples require a score, correction, and personalized feedback, motivating rubric constraints and the critic. Thus MARLET routes by grounding need rather than treating every sample as query-only retrieval.

The modules are also grounded in prior advantages: adaptive RAG treats retrieval as a decision [6], [8], [9]; ReAct/AutoGen motivate planning and role decomposition [11], [12]; tutoring research motivates learner assessment [2], [16]; tutoring benchmarks motivate rubric-style answer criteria [19], [20]; and long-context analyses plus MMR motivate compact relevance-diversity context packing [21], [22].

The design principle is simple: retrieve when external evidence is needed, and otherwise spend context on learner state and answer criteria. Validation keeps the tutor/critic backbone, corpus, reranker, packer, and response budget fixed so that the classical RAG comparison tests how context is assembled before the same tutor writes.

A. Supervisory Routing

The supervisor is the framework’s decision-making component. It estimates whether a sample primarily needs external evidence, pedagogical coordination, rubric feedback, planning, or dialogue-sensitive tutoring. The reported runs use transparent heuristics rather than a trained router. For a standardized benchmark sample x , the router first builds a normalized text blob $b(x)$ from the question, optional context, rubric text, and dialogue.

The first routing quantity is a cue-hit rate. For each cue phrase u , define $m(u, x) = 1$ if u appears in $b(x)$, and $m(u, x) = 0$ otherwise. For a cue family G , the router

computes

$$\kappa_G(x) = \frac{\sum_{u \in G} m(u, x)}{|G|}, \quad (1)$$

where $|G|$ is the number of cue phrases in that family. Thus $\kappa_G(x)$ is just the fraction of cues that were observed: 0 means no cue matched, and 1 means every cue matched. The cue families are evidence (E), coordination (C), rubric (R), planning (P), and tutoring (T).

The cue-hit rate is then converted into a routing score:

$$s_j(x) = \min\{1, \max\{0, \kappa_j(x) + B_j(x) + P_j(d_x)\}\}, \quad (2)$$

where $j \in \{e, c, r, p, t\}$ indexes evidence, coordination, rubric, planning, and tutoring, and $\kappa_j(x)$ means the cue-hit rate for that cue family. The term $B_j(x)$ is a field bonus from visible sample structure: context or image fields increase the evidence score, rubric fields increase the rubric score, and longer dialogue increases the tutoring score. The term $P_j(d_x)$ is a dataset prior from the dataset label d_x ; for example, evidence-grounded datasets receive an evidence prior. The outer min/max only keeps the score between 0 and 1. These scores are not probabilities; they are transparent control signals used to decide which modules should run.

The router then assigns a pedagogical regime: lesson planning, rubric feedback, adaptive tutoring, or evidence-grounded reasoning. If the adapter supplies a regime hint, the supervisor uses it; otherwise it selects lesson planning when s_p is strongest and at least 0.42, rubric feedback when $s_r \geq \max\{s_t, 0.35\}$, adaptive tutoring when $s_t \geq 0.35$, and evidence-grounded reasoning otherwise. Retrieval is controlled by the binary gate

$$g(x) = \begin{cases} 1, & s_e(x) \geq 0.35 \text{ or regime}(x) = \text{EG}, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

Here $s_e(x)$ is the evidence score and $\text{regime}(x) = \text{EG}$ means evidence-grounded reasoning. In words, retrieval is requested when the sample looks evidence-heavy or the regime itself is evidence-grounded. If $g(x) = 0$, the framework still builds the tutoring brief but skips retrieval. If no chunks have been retrieved and the evidence score is at least 0.45, one fallback retrieval pass is issued using the raw question. Thus the router constrains both how much pedagogical coordination to activate and whether the retriever should be used.

B. Specialist Agents and Shared Context

All implemented mechanisms operate over the same tutoring prompt, inferred learning state, rubric summary, retrieved evidence, and draft answer. The tutor and the critic are held constant across all implemented mechanisms: the tutor writes from the supplied context, and the route-controlled critic may revise the draft when the regime calls for adaptive tutoring. The only moving parts are the upstream context builders. In the reported configuration, planner, diagnoser, and rubric are deterministic heuristics. Their roles are asymmetric. The planner alone is retrieval-facing: it emits up to three

deduplicated queries from the raw question, extracted key terms, dataset cueing, and the first rubric items. The diagnoser and rubric module are tutor-facing: they summarize transient learning state and answer criteria for the tutor prompt, but they do not alter retriever scoring. In this release, learning state is inferred from the current question and dialogue rather than from a persistent profile store.

The tutor itself is the LLM-driven agent that produces the final response. It is dataset-aware but architecturally fixed: its prompt includes the dataset name, question, recent dialogue, inferred learning state, operating plan, rubric summary, and either retrieved evidence or inline benchmark context. It is instructed to be correct, pedagogically useful, concise, and to end with a next step or check-for-understanding when appropriate. Retrieved evidence must be cited with `[doc_id]` markers. EduBench consensus rows require a JSON object with a score, scoring details, and personalized feedback; TutorEval and the other tutoring-style profiles use instructional prose. This design isolates the effect of changing the retrieval context while leaving the downstream writer unchanged; personalization reaches retrieval mainly through planner query formulation, not through profile-aware reranking.

C. Hybrid Retrieval, Reranking, and Context Packing

The retrieval stack is intentionally lightweight and also follows the codebase exactly. The following equations are scoring rules, not learned models: a larger score means the chunk is ranked higher for the current query. For a query q and candidate chunk c , the hybrid index first computes

$$\text{score}_0 = 0.58 k_w + 0.17 k_{ch} + 0.25 k_\ell + \beta, \quad (4)$$

where all terms are computed for the current (q, c) pair. Here k_w is word-level TF-IDF similarity, k_{ch} is character 3–5 gram TF-IDF similarity, and k_ℓ is the similarity in the truncated-SVD latent space. The weights state how much the released retriever trusts each signal: word similarity contributes most, latent semantic similarity is second, and character similarity helps with spelling variants and short phrases. The query boost β adds 0.01 per overlapping title token and, in citation-seeking mode, an additional 0.025 for chunks whose source type is reference, lecture, or document. When multiple planner queries are issued, the retriever merges all returned candidates and deduplicates them by chunk identifier before reranking.

In the released paper configuration, no trained reranker is loaded, so the heuristic reranker score is

$$\text{score}_1 = 0.55 \text{score}_0 + 0.20 o_t + 0.10 o_h + 0.06 m_p + 0.05 o_n + 0.04 \rho, \quad (5)$$

where all terms are again computed for the current (q, c) pair. The variables o_t , o_h , m_p , and o_n denote token overlap, title overlap, exact phrase-match score, and numeric overlap, and ρ is a brevity prior. This reranker

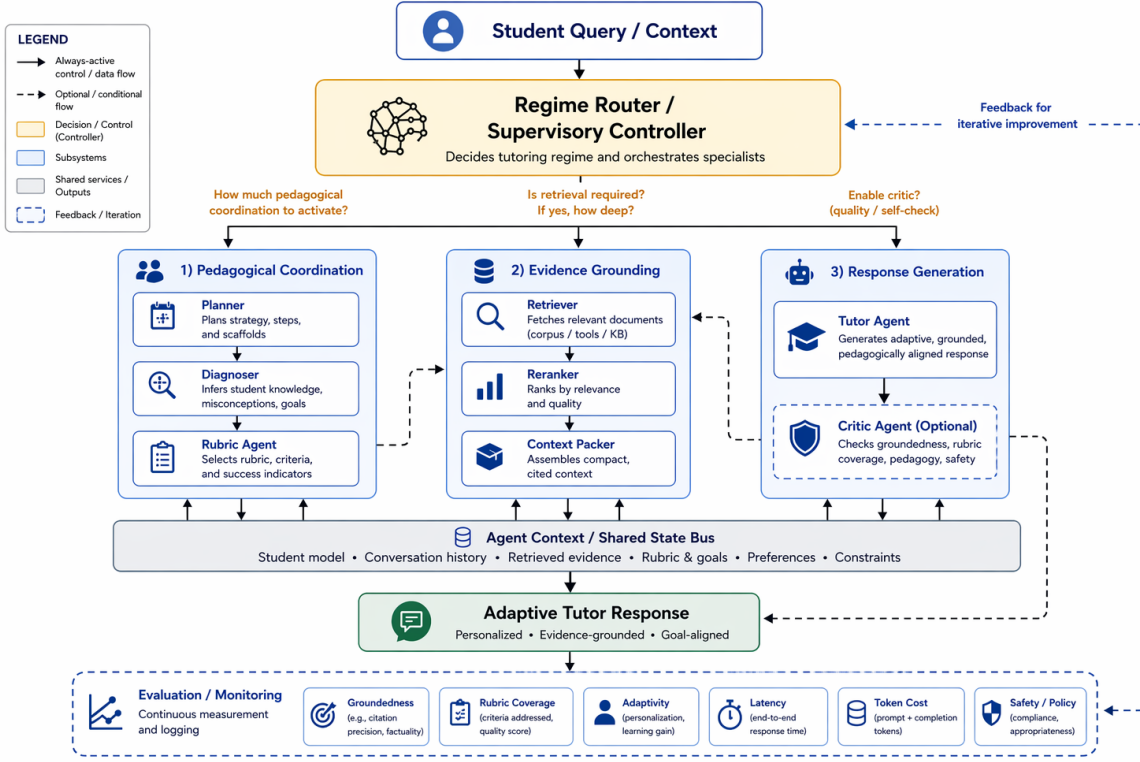


Fig. 1. Main framework of MARLET. The supervisory router scores the sample, assigns the pedagogical regime, and decides whether retrieval is needed before planner, diagnoser, and rubric modules assemble the tutoring brief.

favors chunks that match the question text, title, exact phrases, and numbers while mildly preferring concise chunks. Once planner queries are merged, reranking remains question-centric and does not consume the learner summary directly.

Final context packing balances relevance against redundancy:

$$c^* = \arg \max_{c \in \mathcal{C} \setminus S} \lambda \text{score}_1(q, c) - (1 - \lambda) \max_{c' \in S} \text{overlap}(c, c'), \quad (6)$$

where c^* is the next selected chunk, $\mathcal{C}$ is the reranked candidate pool, S is the set of already packed chunks, and $\text{overlap}(c, c')$ is the redundancy term between two chunks. The first term rewards relevance to the query; the second term penalizes selecting another chunk that repeats content already in S . With $\lambda = 0.68$, a four-chunk cap, and a 6000-character context budget, this keeps the comparison focused on retrieval quality rather than on simply enlarging the context window.

D. Evaluation Protocol

Evaluation is dataset-aware, and all reported quality scores are automatic proxies rather than official benchmark or human grades. Higher is better for quality metrics; lower is better for latency, tokens, and calls. Following RAG evaluation work [10], we separate answer overlap, rubric fulfillment, grounding, and resource cost. For a generated answer y and reference r , Token-F1 is $2PR/(P+R)$, where P is the fraction of generated tokens

also appearing in r and R is the fraction of reference tokens recovered by y . Rubric coverage is the covered-item ratio over the benchmark rubric: an item counts as covered only when y contains a near-verbatim span or at least 55% of the item’s tokens plus one content-bearing rubric token. This prevents long generic answers from receiving credit merely for common words. Latency is wall-clock time per sample; total tokens are prompt plus completion tokens; backbone calls count logged LLM requests.

For EduBench, EB12D is the mean of 12 normalized checks in three groups: scenario adaptation, factual/reasoning behavior, and pedagogical application. These checks cover output-format compliance, supportive non-negative tone, question relevance, use of scenario details, score agreement, domain/rubric grounding, reasoning quality, correction cues, clarity, motivation, personalization, and higher-order prompting. EduAlign is narrower: it measures only numeric grading agreement, $\max(0, 1 - |s(y) - \bar{s}|/100)$, where $s(y)$ is the score extracted from the model output and $\bar{s}$ is the benchmark reference mean. Thus EduAlign can decrease even when explanatory feedback or rubric coverage improves.

For TutorEval, tutor-hit measures teaching-point coverage. If a sample has K expert key points, each point receives full credit when it is stated explicitly or at least 70% of its tokens appear in y ; 25–70% overlap receives linearly scaled partial credit; lower overlap receives zero. The final tutor-hit is the average over the

K points. EduBench is therefore treated as structured educational judgment, while TutorEval is treated as chapter-grounded tutoring. Off-profile metrics remain zeros in raw logs but are omitted from paper tables. The workload audit $U = T + 160Q_r + 90C_r + 240N_a + 30E_v$ uses T for total tokens, Q_r for retrieval queries, C_r for retrieved chunks, N_a for materialized agent outputs, and E_v for trace events. The coefficients are fixed audit weights for efficiency inspection only; U is not a training objective.

IV. Validation Experiments

We validate the framework on two public educational benchmarks. TutorEval [1] contains 834 expert-written science tutoring questions paired with long STEM chapters and teaching key points, so responses must explain, disambiguate, and stay chapter-grounded. EduBench [18] covers nine educational scenarios, including question answering, error correction, personalized support, emotional support, grading, teaching-material generation, and personalized content creation. Its rows contain prompts, scenario metadata, and reference model predictions; our adapter converts them into consensus-style supervision through score statistics and rubric terms. For reporting, we use the 1562-example four-way EduBench intersection completed by all forced architectures.

The primary validation compares classical RAG with MARLET under matched Qwen3.5-4B infrastructure, reasoning budget 512, temperature 0.15, response cap 900, shared corpus, shared reranker, and fixed retrieval hyperparameters; a smaller Qwen3.5-27B-FP8 replication checks backbone sensitivity. Both systems run on the same samples. In classical RAG, retrieval uses the original benchmark fields, including any supplied learner state or context; the tutor receives those fields and chunks, but not MARLET’s planner, diagnosis, or rubric brief. This is the intended contrast: the baseline tests retrieve-then-read grounding, while the proposed framework tests explicit context construction plus conditional retrieval.

The validation is fair at the infrastructure level but intentionally not identical at context construction: giving classical RAG the planner, diagnoser, and rubric outputs would convert it into the proposed framework, while withholding them from MARLET would remove the claimed contribution. We therefore report quality and resource outcomes together.

A. Framework Validation

Table II reports paired matched-sample 4B comparisons with bootstrap confidence intervals and permutation p-values. On TutorEval, MARLET raises Token-F1, tutor-hit, and rubric coverage while using fewer tokens and fewer backbone calls; latency trends lower but is not reliable ($p=0.21$). Table III reports the 27B replication.

TABLE II

4B quality and 4090 resource summary. Quality is higher-better; runtime, tokens, and calls are lower-better.

Data	Metric	RAG	Ours	Δ
TutorEval	Token-F1	0.112	0.115	+0.004*
	TE hit	0.938	0.957	+0.019**
	Rubric	0.919	0.944	+0.026**
	Latency (s)	51.90	49.11	−2.79
	Tokens	3975	3490	−484***
	Calls	1.84	1.56	−0.28
EduBench	EB12D	0.367	0.398	+0.031***
	Rubric	0.705	0.891	+0.186***
	EduAlign	0.166	0.126	−0.040**
	Latency (s)	19.83	13.90	−5.92***
	Tokens	4020	2226	−1794***
	Calls	1.97	1.48	−0.49***

*** $p<10^{-4}$, ** $p<0.01$, * $p<0.05$. Runtime is mean wall-clock seconds on the 4090 server.

TABLE III

27B matched-slice replication ($\Delta = \text{Ours} - \text{RAG}$).

Data	n	Quality Δ	Resource Δ
TutorEval	200	F1 −0.000, hit +0.001, rub. −0.002	latency −11.8 s***, tokens −992***, calls −0.56***
EduBench	87	EB12D +0.032**, align +0.210***, −0.063	rub. latency −22.1 s***, tokens −1861***, calls −0.57***

*** $p<10^{-4}$, ** $p<0.01$. Latency Δ is wall-clock seconds/sample on the 4090 server.

B. Trade-offs and Ablations

For TutorEval, the 95% confidence intervals are [0.0002, 0.007] for Token-F1, [0.006, 0.032] for tutor-hit, and [0.006, 0.044] for rubric coverage. For EduBench, they are [0.027, 0.036] for EB12D, [0.174, 0.198] for rubric coverage, and [−0.069, −0.011] for EduAlign; cost intervals are [−6702, −5248] ms for latency, [−1847, −1741] tokens, and [−0.52, −0.46] calls. Classical RAG keeps a small EduAlign edge, but the frozen hybrid skips retrieval on every EduBench sample, so these prompts are usually better served by learner- and rubric-centered coordination than by external lookup.

The framework can use more modules than classical RAG, so resource cost must be reported. Here, scoped planner queries and the retrieval gate reduce token use. Two 4B ablations on a shared TutorEval slice ($n=100$) isolate the mechanism: forcing retrieval hurts Token-F1 (−0.003), tutor-hit (−0.015), and rubric coverage (−0.025), while disabling the shared critic shows that the main gain does not come from critic revision. The 27B replication shows the same resource pattern; TutorEval quality is essentially tied, while EduBench keeps the EB12D/rubric advantage and EduAlign trade-off.

These results clarify the intended use of MARLET: it is not a universal replacement for retrieval, but a controller for deciding what grounding a tutoring sample needs. Chapter-grounded samples still need evidence, whereas learner-profile, rubric, or scenario-heavy samples may be better served by learner-state and answer-criteria summaries than by mandatory topical retrieval.

V. Limitations

The study validates a framework rather than a final tutoring product. It uses automatic evaluation on English benchmark outputs and infers learner state only from visible prompt/dialogue fields. Real deployment would require privacy safeguards, bias checks, data governance, human oversight, and downstream learning-gain studies.

VI. Conclusion

MARLET reframes grounding for educational tutoring as supervised context construction rather than query-only passage lookup. Under shared infrastructure, it improves fixed-tutor quality and efficiency over classical RAG, showing that learner state, rubric constraints, and conditional evidence should be coordinated before the tutor writes. Future work should add stronger controller baselines, human and multi-turn evaluation, learned router calibration, profile-aware reranking, and privacy-preserving learner-state inference.

References

- [1] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, “Language models as science tutors,” arXiv:2402.11111, 2024. [Online]. Available: <https://arxiv.org/abs/2402.11111>
- [2] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, “AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems,” arXiv:2601.04219, 2026. [Online]. Available: <https://arxiv.org/abs/2601.04219>
- [3] E. Dinan, S. Roller, K. Shuster, A. Fan, M. Auli, and J. Weston, “Wizard of Wikipedia: Knowledge-powered conversational agents,” in Proceedings of the International Conference on Learning Representations, 2019. [Online]. Available: <https://arxiv.org/abs/1811.01241>
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
- [5] N. Thakur, N. Reimers, A. Rüklé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2021. [Online]. Available: <https://openreview.net/forum?id=wCu6T5xFjeJ>
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in International Conference on Learning Representations, 2024. [Online]. Available: <https://arxiv.org/abs/2310.11511>
- [7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” arXiv:2401.15884, 2024. [Online]. Available: <https://arxiv.org/abs/2401.15884>
- [8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 7036–7050. [Online]. Available: <https://aclanthology.org/2024.naacl-long.389/>
- [9] Y. Wang, L. Wei, and H. Ling, “Retrieval as a decision: Training-free adaptive gating for efficient RAG,” arXiv:2511.09803, 2026. [Online]. Available: <https://arxiv.org/abs/2511.09803>
- [10] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, “RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.08067>
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in International Conference on Learning Representations, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv:2308.08155, 2024. [Online]. Available: <https://arxiv.org/abs/2308.08155>
- [13] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for “mind” exploration of large language model society,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.17760>
- [14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.00352>
- [15] T. Nguyen, P. Chin, and Y.-W. Tai, “MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning,” arXiv:2505.20096, 2025. [Online]. Available: <https://arxiv.org/abs/2505.20096>
- [16] C. Borchers and T. Shou, “Can large language models match tutoring system adaptivity? A benchmarking study,” in Artificial Intelligence in Education. Springer Nature Switzerland, 2025, pp. 407–420. [Online]. Available: <https://arxiv.org/abs/2504.05570>
- [17] K. K. Maurya, K. A. Srivatsa, K. Petukhova, and E. Kochmar, “Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors,” in Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025. [Online]. Available: <https://aclanthology.org/2025.naacl-long.57/>
- [18] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, “EduBench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios,” arXiv:2505.16160, 2025. [Online]. Available: <https://arxiv.org/abs/2505.16160>
- [19] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, “MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors,” in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 204–221. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.11/>
- [20] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, “TutorBench: A benchmark to assess tutoring capabilities of large language models,” arXiv:2510.02663, 2025. [Online]. Available: <https://arxiv.org/abs/2510.02663>
- [21] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” Transactions of the Association for Computational Linguistics, vol. 12, 2024. [Online]. Available: <https://aclanthology.org/2024.tacl.1.9/>
- [22] J. Carbonell and J. Goldstein, “The use of mmr, diversity-based reranking for reordering documents and producing summaries,” in Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 1998, pp. 335–336.